

Apex Bot — Deriv Trading Logic

JavaScript / Node.js source code

```
'use strict';

require('dotenv').config();
const WebSocket = require('ws');

const API_BASE = 'https://api.derivws.com';
const APP_ID = process.env.DERIV_APP_ID;
const AUTH_TOKEN = process.env.DERIV_TOKEN;
const ACCOUNT_ID = process.env.DERIV_ACCOUNT_ID;
const SYMBOL = '1HZ100V';

const BASE_STAKE = 1;
let current_stake = BASE_STAKE;
const MAX_STAKE = Number(process.env.MAX_STAKE || 32);

let ws;
let requestId = 1;
const pending = new Map();

function nextRequestId() { return requestId++; }

async function getAuthenticatedWebSocketUrl() {
  const response = await fetch(
    `${API_BASE}/trading/v1/options/accounts/${ACCOUNT_ID}/otp`,
    {
      method: 'POST',
      headers: {
        Authorization: `Bearer ${AUTH_TOKEN}`,
        'Deriv-App-ID': APP_ID,
        'Content-Type': 'application/json'
      }
    }
  );
  if (!response.ok)
    throw new Error(`OTP request failed: ${response.status}`);

  const result = await response.json();

  if (!result.data?.url)
    throw new Error('No authenticated WebSocket URL returned');

  return result.data.url;
}

function send(request) {
  return new Promise((resolve, reject) => {
    if (!ws || ws.readyState !== WebSocket.OPEN) {
      reject(new Error('WebSocket is not connected'));
      return;
    }

    const req_id = nextRequestId();

    pending.set(req_id, { resolve, reject });

    ws.send(JSON.stringify({ ...request, req_id }));
  });
}

/* MARTINGALE MANAGEMENT */

function handleContractResult(contract) {
  const status = String(contract.status || '').toLowerCase();

  console.log(
    `Contract ${contract.contract_id} finished with status: ${status}`
  );

  if (status === 'won') {
    current_stake = BASE_STAKE;
    console.log(`WIN -> current_stake reset to ${current_stake}`);
  } else if (status === 'lost') {
    const nextStake = current_stake * 2;

    if (nextStake > MAX_STAKE) {
      console.log(
        `LOSS -> next stake ${nextStake} exceeds MAX_STAKE ` +
        `${MAX_STAKE}. Resetting to ${BASE_STAKE}.`
      );
      current_stake = BASE_STAKE;
    } else {
      current_stake = nextStake;
      console.log(`LOSS -> next current_stake = ${current_stake}`);
    }
  }
}

/* BUY RISE CONTRACT */

async function buyRiseOneTick() {
  const stakeForThisTrade = current_stake;

  console.log(
```

```

        `Preparing RISE trade with stake ${stakeForThisTrade}`
    );
}

try {
    const proposalResponse = await send({
        proposal: 1,
        amount: stakeForThisTrade,
        basis: 'stake',
        contract_type: 'CALL',
        currency: 'USD',
        duration: 1,
        duration_unit: 't',
        underlying_symbol: SYMBOL
    });

    if (proposalResponse.error) {
        console.error('Proposal error:',
            proposalResponse.error.message);
        return;
    }

    const proposalId = proposalResponse.proposal?.id;
    const askPrice = Number(
        proposalResponse.proposal?.ask_price
    );

    if (!proposalId || !Number.isFinite(askPrice)) {
        console.error(
            'Invalid proposal response:', proposalResponse
        );
        return;
    }

    const buyResponse = await send({
        buy: proposalId,
        price: askPrice
    });

    if (buyResponse.error) {
        console.error('BUY ERROR:',
            buyResponse.error.message);
        return;
    }

    const contractId = buyResponse.buy?.contract_id;

    if (!contractId) {
        console.error('No contract ID returned');
        return;
    }

    console.log(
        `BOUGHT RISE | Contract=${contractId} | ` +
        `Stake=${stakeForThisTrade}`
    );

    await send({
        proposal_open_contract: 1,
        contract_id: contractId,
        subscribe: 1
    });

} catch (error) {
    console.error('Trade execution error:', error.message);
}

}

/* TICK HANDLER */

function handleTick(tick) {
    const quote = String(tick.quote);
    const digitsOnly = quote.replace(/\D/g, '');

    if (!digitsOnly.length) return;

    const lastDigit = Number(
        digitsOnly[digitsOnly.length - 1]
    );

    console.log(
        `Tick ${quote} | Last digit=${lastDigit} | ` +
        `Current stake=${current_stake}`
    );

    if (lastDigit % 2 === 0) {
        console.log(`EVEN ${lastDigit} -> BUY RISE`);
        void buyRiseOneTick();
    }
}

/* WEBSOCKET */

function connect(wsUrl) {
    ws = new WebSocket(wsUrl);

    ws.on('open', async () => {
        console.log('Connected to Deriv');

        try {
            await send({

```

```

        ticks: SYMBOL,
        subscribe: 1
    });

    console.log(`Subscribed to ${SYMBOL}`);
} catch (error) {
    console.error('Subscription error:', error.message);
}
});

ws.on('message', raw => {
    let message;

    try {
        message = JSON.parse(raw.toString());
    } catch {
        console.error('Invalid JSON received');
        return;
    }

    if (message.req_id && pending.has(message.req_id)) {
        const request = pending.get(message.req_id);
        pending.delete(message.req_id);
        request.resolve(message);
    }

    if (message.msg_type === 'tick') {
        handleTick(message.tick);
    }

    if (message.msg_type === 'proposal_open_contract') {
        const contract = message.proposal_open_contract;

        console.log(
            `Contract update | ID=${contract.contract_id} | ` +
            `Status=${contract.status} | Profit=${contract.profit}`
        );

        if (
            contract.is_sold &&
            (contract.status === 'won' || contract.status === 'lost')
        ) {
            handleContractResult(contract);
        }
    }

    if (message.error) {
        console.error(
            'Deriv API error:', message.error.message
        );
    }
});

ws.on('error', error => {
    console.error('WebSocket error:', error.message);
});

ws.on('close', () => {
    console.log('Connection closed. Reconnecting...');
    setTimeout(() => connect(wsUrl), 3000);
});
}

/* START */

async function start() {
    try {
        console.log('Starting Apex Bot...');
        console.log(`Initial stake: ${BASE_STAKE}`);
        console.log(`Maximum stake: ${MAX_STAKE}`);

        const wsUrl = await getAuthenticatedWebSocketUrl();
        connect(wsUrl);

    } catch (error) {
        console.error('Startup error:', error.message);
        setTimeout(start, 5000);
    }
}

process.on('SIGINT', () => {
    console.log('\nStopping bot...');
    if (ws) ws.close();
    process.exit(0);
});

start();

```